Práctica 3

Objetivos:

Seguir trabajando con clases y con matrices utilizando la librería *numpy* para realizar operaciones básicas como creación, modificación, visualización, suma y resta de matrices en 1D y 2D.

Instrucciones:

Haga uso de anotaciones, `docstring` y la convención de nombres explicada en clase de teoría para nombrar las clases, funciones, variables y métodos. Represente la clase en notación UML, siguiendo cualquiera de los formatos descritos en la práctica 2.

```
import numpy as np

class MatrizFloat:
    """
    Clase que representa una matriz dinámica 1D/2D.

    Atributos de clase:
        TIPO_DATO  #Tipo de dato por defecto (np.float32)

    Atributos de instancia (protegidos):
        matriz   # Almacena la matriz (utilice numpy)
        nfil     # Almacena número de filas de la matriz (int)
        ncol     # Almacena número de columnas de la matriz (int)
    """

    def __init__(...):
        """
        Inicializador. No recibe parámetros adicionales.
        Inicializa el atributo matriz con None y los atributos
        nfil y ncol a 0
        """
        pass

    def matriz_identidad(...):
        """
          Método de clase que crea y devuelve una nueva
          instancia de MatrizFloat configurada como matriz
          identidad de tamaño n x n

        Parámetros:
            El tamaño n de la matriz
```

```
        Devuelve:
            Nueva instancia con matriz identidad creada
            (MatrizFloat)
        """
        #NOTA: Utilice "eye" de numpy para crear la matriz
               identidad
        pass

    def mismas_dimensiones(...)
        """
        Método para comparar dimensiones.

        Parámetros:
            m1   #Primera matriz a comparar (MatrizFloat)
            m2   #Segunda matriz a comparar (MatrizFloat)

        Devuelve:
            bool: True si ambas matrices tienen el mismo número de
                  filas y columnas, y False en caso contrario
                  (o si alguna no está creada)
        """
        pass

     def crear_matriz2D(...):
        """
        Método para crear una matriz bidimensional de ceros.
        Asigna valores de filas y columnas según parámetros.
        Parámetros:
            nFil:   Número de filas (int)
            nCol:   Número de columnas (int)
        """
        pass

     def crear_matriz1D(...):
        """
        Método para crear una matriz unidimensional de ceros.
        Usa crear_matriz2D para asignar 1 fila y n columnas.
        Parámetros:
            nElementos:  Número de elementos de la fila (int)
        """
        pass

     def existe(...):
        """
```

```
        Método que verifica si matriz está creada y no está vacía
        Devuelve:
            True si existe, de lo contrario False
        """
        pass

    def introducir(...):
        """
        Método que solicita por consola el valor numérico decimal
        para cada posición (i, j) de la matriz.

        Da error si se intenta introducir datos sin haber creado la
        matriz previamente.
        """
        pass


    def mostrar(...):
        """
        Método para mostrar las dimensiones y los elementos de la
        matriz. Si la matriz no existe (existe() == False), muestra
        un mensaje advirtiéndolo
        """
        pass
    # ============================================================
    # OPERACIONES ALGEBRAICAS
    # ============================================================

    def sumar_matrices(...):
        """
        Método que suma la matriz actual con otra matriz.

        Parámetros:
            otra_matriz: objeto de MatrizFloat con la matriz a sumar

        Devuelve:
            Un nuevo objeto MatrizFloat con la matriz resultante de
            la suma.
        """
        #Nota: Las dimensiones de ambas matrices deben coincidir.

        pass

    def restar_matrices(...):
        """
```

```
       Método que resta la matriz actual con otra matriz.

       Parámetros:
          otra_matriz: objeto de MatrizFloat con la matriz a restar

       Devuelve:
          Un nuevo objeto MatrizFloat con la matriz resultante de
          la resta.
       """
       #Nota: Las dimensiones de ambas matrices deben coincidir.

       pass
```

Se implementarán las siguientes funciones que podrán reutilizarse en prácticas futuras:

```
def leer_int(...):
    """
    Función auxiliar que lee un número entero del teclado.
    Si se introduce un valor no válido, se solicita de nuevo.
    """
    Parámetros:
        mensaje   # El mensaje que se muestra al usuario
                  # solicitando la entrada (str). Por defecto
                  # "Introduce un número entero: "

    Devuelve:
        int: El valor entero introducido por el usuario.

    pass

def leer_float(...):
    """
    Función auxiliar que solicita al usuario un número decimal.
    Si se introduce un valor no válido, se solicitar de nuevo.

    Parámetros:
        mensaje   # El mensaje que se muestra al usuario
                  # solicitando la entrada (str). Por defecto
                  # "Introduce un número decimal: "
    Devuelve:
        El valor decimal introducido por el usuario (float)
    """
    pass
```

```
def crear_menu(...):
    """
    Función que muestra un menú de opciones y solicita al usuario
    que seleccione una opción válida.

    Parámetros:
        opciones_menu  # lista de opciones a mostrar en el menú
                       # (list)


    Devuelve:
        El número de opción seleccionado por el usuario (int)
    """
    pass
```

Programa principal

El programa incluirá un menú principal con las siguientes opciones:

1. Construir matriz 1D: permite crear un vector unidimensional con un número definido de elementos.
2. Construir matriz 2D: permite crear una matriz bidimensional definiendo el número de filas y columnas.
3. Construir matriz identidad: solicita al usuario el tamaño (n) y crea una matriz identidad nXn mediante el método de clase `matriz_identidad()`.
4. Introducir matriz: permite al usuario ingresar los valores para la matriz creada.
5. Mostrar matriz: muestra los valores actuales de la matriz.
6. Operaciones con matrices: Lleva al usuario a un submenú donde se pueden realizar las operaciones de suma y resta de matrices.
7. Terminar: finaliza el programa.

Para la opción 6 del menú principal, habrá un submenú de Operaciones con matrices con las siguientes opciones:

1. Sumar matrices: solicita al usuario una segunda matriz y suma ambas matrices, mostrando el resultado.
2. Restar matrices: solicita al usuario una segunda matriz y resta ambas matrices, mostrando el resultado.
3. Volver al menú principal: regresa al menú principal.

Estructura del programa

- Cree un archivo llamado *matriz.py* para la clase `MatrizFloat` y las funciones relacionadas.
- Cree un archivo llamado *utils.py* para el menú y funciones de lectura de datos.
- Implemente una función `main()` que ejecute el menú.